

AI-Powered Resume Screening and Job Matching System

Field	Value
Document Version	1.0
Prepared By	Sujal Pawar
Date	23-Jul-2026

1. Introduction

1.1 Purpose

- Define complete requirements for AI-Powered Resume Screening and Job Matching System
- Automates resume screening against job descriptions using ML techniques
- Provides ranked candidate list with skill gap analysis and exportable reports

1.2 Project Scope

- Bulk resume upload from a folder + job description input
- ML-based matching score (TF-IDF + Cosine Similarity) with matched skills, missing skills and percentage
- Skill gap analysis (skills required by job but missing in resume)
- Experience level classification based on extracted years of experience
- Ranked candidate report with detailed columns + CSV/Excel export
- Robust error handling for invalid / corrupt / empty files
- Secure admin login with bcrypt hashed passwords

2. Overall Description

2.1 Product Perspective

- No web server or internet dependency for core features
- Local storage via SQLite database

2.2 User Characteristics

- Primary users: HR personnel / Recruiters
- Basic computer and CLI knowledge only - no coding skills required

2.3 Assumptions and Dependencies

- Resumes provided in PDF or plain text format
- Python 3.8+ and required libraries pre-installed
- Job description provided as plain text
- Single-machine execution (no server deployment in current version)

3. Functional Requirements

ID	Requirement	Description
FR-1	Admin Login	Username/password login before access. Passwords stored using bcrypt hashing (never plain text).
FR-2	Bulk Resume Upload	Process multiple resumes from a user-specified folder in one go.
FR-3	Resume Parsing	Extract and clean text from PDF (pdfplumber/PyPDF2). Includes basic skill keyword extraction.
FR-4	Contact Info Extraction	Auto-detect email and phone number from resume text using Regex.
FR-5	Job Description Input	Accept job title and full job requirements as plain text.
FR-6	ML-Based Matching	TF-IDF + Cosine Similarity. Output: match percentage, top matched skills, and missing skills.
FR-7	Skill Gap Analysis	List skills present in job description but missing from the candidate resume.
FR-8	Experience Classification	Extract approximate years of experience and classify as Fresher (0-1), Junior (1-3), or Senior (3+).
FR-9	Candidate Ranking	Rank by matching score. Table shows Rank, Name, Email, Experience, Score %, Matched Skills, Missing Skills.
FR-10	Report Export	Export ranked candidate list as CSV and Excel files.
FR-11	Data Storage	Store candidates, jobs, scores, status and timestamps in SQLite.
FR-12	File Validation & Error Handling	Validate file type. Gracefully skip corrupt/empty/invalid files and show clear messages without crashing.
FR-13	Password Security	Store admin passwords using bcrypt hashing. Never store plain-text passwords.

4. Non-Functional Requirements

Type	Requirement
Performance	Process up to 20 resumes within a few seconds on standard hardware.
Reliability	Graceful handling of invalid, corrupt, empty or non-PDF files. System must not crash.
Security	Login required for all data access. Passwords stored with bcrypt hashing.
Maintainability	Modular code - separate modules for authentication, parsing, matching, skills, database and reporting.
Portability	Runs on any OS (Windows / Linux / macOS) with Python 3.8+.

5. System Requirements

5.1 Hardware

- Processor: Dual-core or higher
- RAM: 4 GB minimum
- Storage: 500 MB free space

5.2 Software

- OS: Windows / Linux / macOS
- Python 3.8+
- Libraries: pdfplumber / PyPDF2, scikit-learn, pandas, sqlite3, bcrypt

6. Technologies Used

Technology	Purpose
Python	Core language for the entire application
pdfplumber / PyPDF2	Extract and clean text from resume PDFs
scikit-learn (TF-IDF + Cosine Similarity)	ML-based resume-job matching and scoring
SQLite	Local storage of users, candidates, jobs, scores and status
Pandas	Data handling and CSV / Excel export
bcrypt	Secure password hashing for admin authentication

7. System Flow

7.2 System Flow Diagram



1. Admin Login (with hashed password verification)
2. Upload Resumes (Bulk) + Enter Job Description
3. File Validation → Resume Parsing (text + skills + experience years)
4. Extract Contact Info (Email, Phone)
5. TF-IDF + Cosine Similarity Matching → Score % + Matched Skills
6. Skill Gap Analysis + Experience Classification
7. Rank Candidates (detailed informative table)
8. Store results in SQLite (with status & timestamps)
9. Display Report / Export CSV-Excel

8. Database Design

Table	Key Fields
Users	user_id, username, password_hash, role
Candidates	candidate_id, name, email, phone, experience_years, experience_level, resume_path
Jobs	job_id, job_title, job_description, status
Scores	score_id, candidate_id, job_id, matching_score, matched_skills, missing_skills

9. Future Scope

- GUI using Tkinter or a full web dashboard
- Advanced NLP (NER) for more accurate skill and entity extraction
- Automated email / notification to shortlisted candidates
- Support for multiple simultaneous job postings
- Semantic matching using Sentence Transformers / embeddings
- Role-based access (Admin vs Recruiter) with final permissions